```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BST:

    def insert(self, root, data):
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left, data)

        elif data > root.data:
            root.right = self.insert(root.right, data)

        else:
            print("Duplicate value not inserted")

        return root

    def search(self, root, data):
        if root is None:
            return False

        if root.data == data:
            return True

        if data < root.data:
            return self.search(root.left, data)

        return self.search(root.right, data)

    def find_min(self, root):
        while root.left:
            root = root.left
        return root

    def delete(self, root, data):
        if root is None:
            return None

        if data < root.data:
            root.left = self.delete(root.left, data)
```

```
    elif data > root.data:
        root.right = self.delete(root.right, data)

    else:
        if root.left is None:
            return root.right

        if root.right is None:
            return root.left

        temp = self.find_min(root.right)
        root.data = temp.data
        root.right = self.delete(root.right, temp.data)

    return root

def inorder(self, root):
    if root:
        self.inorder(root.left)
        print(root.data, end=" ")
        self.inorder(root.right)

def preorder(self, root):
    if root:
        print(root.data, end=" ")
        self.preorder(root.left)
        self.preorder(root.right)

def postorder(self, root):
    if root:
        self.postorder(root.left)
        self.postorder(root.right)
        print(root.data, end=" ")

def depth(self, root):
    if root is None:
        return 0

    left = self.depth(root.left)
    right = self.depth(root.right)

    return max(left, right) + 1

def mirror(self, root):
    if root is None:
        return

    root.left, root.right = root.right, root.left
```

```
    self.mirror(root.left)
    self.mirror(root.right)

def copy(self, root):
    if root is None:
        return None

    new_node = Node(root.data)

    new_node.left = self.copy(root.left)
    new_node.right = self.copy(root.right)

    return new_node

def parents(self, root):
    if root:
        if root.left or root.right:
            print("Parent:", root.data, end=" -> ")

            if root.left:
                print("Left:", root.left.data, end=" ")

            if root.right:
                print("Right:", root.right.data)

        self.parents(root.left)
        self.parents(root.right)

def leaf_nodes(self, root):
    if root:
        if root.left is None and root.right is None:
            print(root.data, end=" ")

        self.leaf_nodes(root.left)
        self.leaf_nodes(root.right)

def level_order(self, root):
    if root is None:
        return

    queue = [root]

    while queue:
        node = queue.pop(0)

        print(node.data, end=" ")
```

```
            if node.left:
                queue.append(node.left)

            if node.right:
                queue.append(node.right)


bst = BST()
root = None

while True:

    print("\n\n--- BINARY SEARCH TREE ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Display Traversals")
    print("5. Display Depth")
    print("6. Display Mirror Image")
    print("7. Create Copy")
    print("8. Display Parent Nodes")
    print("9. Display Leaf Nodes")
    print("10. Display Level Wise")
    print("11. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        root = bst.insert(root, data)

    elif choice == 2:
        data = int(input("Enter data to delete: "))

        if bst.search(root, data):
            root = bst.delete(root, data)
            print("Deleted successfully")
        else:
            print("Data not found")

    elif choice == 3:
        data = int(input("Enter data to search: "))

        if bst.search(root, data):
            print("Data found")
        else:
            print("Data not found")
```

```
elif choice == 4:
    print("Inorder:")
    bst.inorder(root)

    print("\nPreorder:")
    bst.preorder(root)

    print("\nPostorder:")
    bst.postorder(root)

elif choice == 5:
    print("Depth:", bst.depth(root))

elif choice == 6:
    bst.mirror(root)

    print("Mirror Image:")
    bst.inorder(root)

elif choice == 7:
    copy_root = bst.copy(root)

    print("Copy created")
    print("Copied Tree:")
    bst.inorder(copy_root)

elif choice == 8:
    print("Parent Nodes:")
    bst.parents(root)

elif choice == 9:
    print("Leaf Nodes:")
    bst.leaf_nodes(root)

elif choice == 10:
    print("Level Wise:")
    bst.level_order(root)

elif choice == 11:
    break

else:
    print("Invalid choice")
```